VIVA PREPARATION NOTES

A Data-Budget-Aware and Explanation-Stable Lightweight Deep Learning Framework for Plant Disease Detection

1. TWO-MINUTE PROJECT EXPLANATION (SAY THIS FIRST)

"Plant diseases reduce crop yield, and farmers cannot always get an expert in time. My project detects plant leaf diseases from a single photo using a **lightweight deep learning model** that also **explains** its decision. I used the PlantVillage dataset of **54,305 images in 38 classes** covering 14 crops. I first audited the data and split it by **leaf groups**, so photos of the same leaf never appear in both training and testing – this avoids data leakage. To study the limited-data situation, I trained on only **40% of the training data (15,240 images)**. I fine-tuned a pretrained **MobileNetV3-Small**, which has only **1.56 million parameters**, with augmentation and class-weighted loss for imbalance. On 8,050 unseen test images it reached **98.30% accuracy** and **97.74% macro F1-score**, and it predicts in about **22 ms on a normal CPU**. I used **Grad-CAM** to show which leaf regions influenced each prediction and measured how stable these heatmaps are. Finally, I built a **Flask web app** with login, image upload, prediction, heatmap, report and history stored in SQLite."

2. PROBLEM, OBJECTIVES AND CONTRIBUTION

- **Problem:** manual inspection is slow and expert-dependent; similar symptoms cause mistakes; large CNNs are heavy; most systems give no explanation and are tested only on the full dataset with a random split.
- **Objectives:** (1) leakage-free split with controlled data budgets, (2) symptom-preserving preprocessing and augmentation, (3) lightweight classifier with balanced performance, (4) Grad-CAM explanations with measured stability, (5) usable web application.
- **My contribution:** not a new network – a complete, reproducible pipeline that combines a leaf-group split, data-budget training, class-weighted learning, quantified explanation stability and a deployed app.

3. EXISTING VS PROPOSED SYSTEM

Existing system	Proposed system
Manual inspection / lab tests – slow, needs experts	Photo-based prediction in ~22 ms
Heavy models (VGG, ResNet, >20 M parameters)	MobileNetV3-Small, 1.56 M parameters, 6.4 MB
Random split – near-duplicate leaves leak into test	Leaf-group split, 0 overlaps
Only overall accuracy reported	Macro precision / recall / F1, per-class F1, confusion matrix
Black-box prediction	Grad-CAM heatmap + stability measured
No user system	Login, report, history (Flask + SQLite)

4. KEY NUMBERS TO REMEMBER

Item	Value	Item	Value
Dataset	PlantVillage, 54,305 images	Accuracy (test)	98.30%
Classes	38 (14 crops: 26 diseases + 12 healthy)	Macro precision	97.74%
Split	70 / 15 / 15 (38,105 / 8,150 / 8,050)	Macro recall	97.78%
Budgets	B20, B40, B60, B80, B100	Macro F1-score	97.74%
Trained on	B40 = 15,240 images	Parameters	1,556,806
Input size	160 × 160 RGB	Model file	6.36 MB
Epochs / batch	5 / 32, AdamW, lr 3e-4, cosine	CPU speed	22.0 ms / image
Training time	30.2 min on CPU	Grad-CAM flip agreement	98.4% (r = 0.947)
Smallest class	Potato healthy (152)	Largest class	Orange Citrus greening (5,507)

5. CORE CONCEPTS IN SIMPLE WORDS

- **CNN:** layers of filters that learn image features automatically – early layers find edges and colours, deeper layers find lesion textures and shapes, the last layer gives class scores.
- **Transfer learning:** start from a model already trained on ImageNet (1.2 M general images) and fine-tune it on leaves – needs less data and trains faster.
- **Why MobileNetV3-Small:** designed for mobile devices – depthwise separable convolutions, inverted residual blocks, squeeze-and-excitation attention and hard-swish activation. Very few parameters, fast on CPU, and still accurate.
- **Depthwise separable convolution:** splits a normal convolution into a per-channel (depthwise) filter and a 1×1 (pointwise) mixing step – about 8–9 times less computation.
- **Data leakage:** when test images are almost identical to training images, accuracy looks higher than it really is. PlantVillage has several photos of the same leaf, so I grouped photos by leaf (40,347 groups) and kept each group in one partition.

- **Data budget:** training on a fixed fraction (20–100%) of the training set while keeping validation and test fixed – shows how the model behaves with limited labelled data.
- **Augmentation:** random crop (60–100% area), horizontal flip, colour jitter (brightness/contrast/saturation ±25%, hue ±0.1) – only on training images, so test data stays clean.
- **Class imbalance and weighted loss:** classes range from 152 to 5,507 images. Each class weight ∝ 1/√(count), normalised to mean 1, so rare classes count more in the loss without over-correcting.
- **Cross-entropy loss:** −w_y · log p(y|x) – penalises low probability for the true class.
- **AdamW + cosine schedule:** Adam optimiser with decoupled weight decay (1e-4); the learning rate falls smoothly from 3e-4 to 0. Best checkpoint chosen by validation macro F1.
- **Accuracy vs macro F1:** accuracy can hide weak small classes. Precision = correct positives / predicted positives; recall = correct positives / actual positives; F1 = their harmonic mean; macro = average over all 38 classes equally.
- **Grad-CAM:** take the last convolution block (576 maps of 5×5), compute the gradient of the predicted class score for each map, average it to get a weight, take the weighted sum, keep positives (ReLU), upsample and overlay as a heatmap. Red = most influential region.
- **Explanation stability:** explain the original and a changed view (flip, +15% brightness, 10° rotation), map back, compare heatmaps with Pearson correlation and top-20% IoU – stable heatmaps are more trustworthy.

6. RESULTS TO EXPLAIN

- Training: loss 0.862 → 0.054, validation accuracy 93.28% → 98.06% in 5 epochs; validation loss stayed close to training loss → **no overfitting**.
- Test: **98.30% accuracy, 97.74% macro F1**; 34 of 38 classes have F1 above 95%. Accuracy ≈ macro F1 means performance is balanced, not driven by large classes.
- Weakest: **Tomato Early blight (86.9%)** and **Corn Cercospora leaf spot Gray leaf spot (89.9%)**. Main confusions: Tomato Late blight ↔ Early blight, Corn Northern Leaf Blight ↔ Gray leaf spot – similar brown lesions.
- Stability (380 test images): brightness – agreement 99.5%, r = 0.993; flip – 98.4%, r = 0.947; rotation 10° – 91.8%, r = 0.809 (weakest, because rotation was not in training augmentation).
- Comparison: Mohanty 2016 GoogLeNet 99.35%, Too 2019 DenseNet-121 99.75%, Pal 2025 Mob-Res 99.47% (3.51 M params) – all used full training data and their own splits; mine uses 40% data, 1.56 M params and a leakage-free split, so it is a fairer, harder test.

7. WEB APPLICATION (FLASK)

- **Flow:** register → login → upload leaf → validate → resize 184, crop 160, normalise → model + Grad-CAM → save to SQLite → result page → report / history.
- **Security:** passwords stored as salted hashes (Werkzeug scrypt), signed session cookies, random server-side file names, 8 MB limit, file-type and image checks, users can open only their own results (others get 404), Jinja2 auto-escaping.
- **Database:** users(id, name, email, password, created_at); predictions(id, user_id, image_file, cam_file, label, confidence, top5, latency_ms, created_at).
- **Low confidence:** below 50% the app warns the user, because the model must always pick one of the 38 known classes.
- **Testing:** 16 test cases – unit (data loading, corrupt file, split overlap, budgets, augmentation, inference, Grad-CAM) and functional (login, wrong password, duplicate email, prediction, invalid files, access control) – all passed.

8. LIKELY VIVA QUESTIONS WITH SHORT ANSWERS

Q1. Why did you choose this project? Early disease detection helps farmers reduce losses; experts are not always available, and a phone photo plus a light model can give instant preliminary screening.

Q2. Why PlantVillage? Public, expert-labelled, 38 classes and 54K images, and the standard benchmark – results can be compared with published work.

Q3. Why not ResNet or VGG? They have 11–138 M parameters. MobileNetV3-Small has 1.56 M, runs in 22 ms on CPU and still gives 98.30% – better suited to low-cost deployment.

Q4. Why only 40% training data? To test the limited-data situation that real farms face. If it works with 40%, it is robust; budgets B20–B100 are prepared for the full study.

Q5. What is the novelty? The combination: leaf-group leakage-free split, data-budget protocol, class-weighted training, quantified Grad-CAM stability and a deployed app – not just reporting accuracy.

Q6. Why 160 × 160 and not 224? Smaller input → about half the computation and faster CPU training/inference; accuracy stayed above 98%.

Q7. How did you avoid overfitting? Pretrained weights, augmentation, weight decay, gradient clipping, checkpoint selection on validation macro F1; validation loss stayed close to training loss.

Q8. Why is the test set never used for tuning? Using it would leak information and inflate results; all choices were made on the validation set and the test set was used once.

Q9. Why only 5 epochs? CPU-only training (~6 min per epoch); with a pretrained backbone, validation macro F1 was already 97.50% and still improving slowly – a cosine schedule finished at epoch 5.

Q10. What does Grad-CAM prove? It shows where the model looked, so we can check it focuses on lesions and not the background. It does not prove biological cause and is coarse (5×5).

Q11. Why measure explanation stability? If a slightly flipped photo gives a totally different heatmap, the explanation cannot be trusted. Correlation 0.947 under flip shows stable explanations.

Q12. Will it work on field photos? Not verified – PlantVillage has plain backgrounds. Field testing and background augmentation are the main future work. The app shows a preliminary result and a low-confidence warning.

Q13. What if the photo is not a leaf? The model still chooses one of 38 classes; low confidence is flagged. An out-of-distribution detector is planned as future work.

Q14. Why macro F1 instead of accuracy? Classes are imbalanced (152 to 5,507). Macro F1 treats every class equally, so a weak small class cannot hide.

Q15. Why does Tomato Early blight score lowest? Its brown concentric lesions look like Late blight and Target Spot; most errors are between these similar diseases.

Q16. How is the class weight calculated? $w = 1/\sqrt{(\text{count})}$, normalised to mean 1 – square root avoids over-correcting very small classes.

Q17. How are passwords stored? Never in plain text – salted hash with Werkzeug (generate_password_hash / check_password_hash).

Q18. Why Flask and SQLite? Flask is light and Python-based, so the PyTorch model loads directly; SQLite needs no server and can later be replaced with MySQL.

Q19. Why PyTorch? Pretrained MobileNetV3 in torchvision and easy forward/backward hooks for Grad-CAM.

Q20. What are the limitations? Lab-style images only, single dataset, one seed and one budget fully trained, no severity estimation, closed set of 38 classes.

Q21. Future enhancements? Field images, rotation/background augmentation, all budgets with repeated seeds, out-of-distribution rejection, mobile app (TFLite/ONNX), Tamil interface.

Q22. Can this replace an expert? No – it is a fast preliminary screening tool; final diagnosis and treatment must come from an agricultural expert.

Q23. What is data leakage in your dataset? Multiple photos of the same leaf; a random split puts them in both train and test. Leaf-group splitting prevents it – 0 overlaps found.

Q24. How long does prediction take? About 22 ms for the model alone and roughly 100 ms including Grad-CAM on a normal CPU.

9. QUICK TIPS FOR THE VIVA

- Start with the 2-minute explanation, then show the app demo (login → upload → result → heatmap → history).
- Always quote numbers exactly: **98.30%** accuracy, **97.74%** macro F1, **1.56 M** parameters, **22 ms**. If unsure, say "as measured on the 8,050-image test set".
- Admit limitations honestly (field images, single dataset) and link them to future work.